

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_98c1f280-b9f\agent-a733dbbdeea0a0b35.jsonl`  
- SHA-256: `d8844a5adfb977f98bd86e83e894531d40d43b73518f60bb7322925a56ef3f80`  
- Source modified: `2026-06-14T06:34:08+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_98c1f280-b9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T06:32:15.321Z)

In the current badminton-highlight-indexer repo (cwd), audit ONE signal/component. Verify and return:

- present: the code exists.
- default_off: it does NOT run in the served/default path by default (e.g. cue_flags default False, default_segmenter is NOT 'fusion', the feature is not weighted by the shipped scorer unless explicitly enabled).
- no_special_casing: the signal enters the DECISION as a confidence-scored / feature COLUMN the learned scorer weights ? NOT a hard if/else branch hard-coded into the decision path.
- retained: NOTHING deletes or disables the code because the n=6 measured ?F1 was non-significant; the producer + feature math still exist and are re-measurable when the corpus grows.

Give evidence as concrete file:line refs with short quotes. List concerns. Be precise and skeptical; read the actual files.

SIGNAL: the PERSON cue. Read backend/pipeline/detectors/person_detector.py and backend/pipeline/detectors/fusion_features.py (compute_person_features, PERSON_FEATURE_NAMES, mean_player_motion, two_sided_motion, shuttle_player_colocation, in_court_ratio).

2. assistant (2026-06-14T06:32:17.698Z)

I'll audit the PERSON cue signal. Let me start by reading the two specified files and pulling the latest repo state.

3. user (2026-06-14T06:32:20.788Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-14T06:32:20.801Z)

```
1      """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.  
2  
3      Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):  
4      the **person cue is a swappable producer**, not logic baked into one segmenter. Today  
5      `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-  
identically);  
6      next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and  
7      later the learned classifier reads its persisted features. Keeping detection here (not  
in  
8      the segmenter) is what makes "add a cue = register a producer" true.  
9  
10     Behaviour is unchanged from the in-lined version ? only the home moved:  
11     - `lazy_load_model`           ? load a permissively-licensed torchvision detector  
12     - `make_tracker`             ? a fresh supervision ByteTrack per chunk  
13     - `detect_and_track`         ? one frame ? confident persons with persistent IDs  
14     - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
```

```

15         with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16         that feed the candidate `stats` JSON and the learned fusion features.
17
18     The model libs (torchvision, supervision) are imported lazily inside the methods, and
19     the
20     detector weights load only on first detection (`lazy_load_model`) ? so importing the
21     segmenter registry never loads a detector model, and tests can mock the seams. (torch +
22     cv2
23     are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
24     machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
25     (ADR-005; ultralytics' AGPL YOLO was dropped).
26     """
27     import logging
28     from typing import Any, Callable, Dict, List, Optional, Tuple
29
30     import cv2
31     import torch
32
33     from backend.utils.geometry import get_velocity_normalised
34
35     logger = logging.getLogger(__name__)
36
37     # torchvision detection models are trained on COCO where the "person" category is
38     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
39     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
40     _PERSON_CLASS_ID = 1
41
42     # Permissively-licensed torchvision detectors selectable via config
43     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
44     _DETECTOR_FACTORIES = {
45         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
46         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
47         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
48     }
49     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
50
51     class PersonDetector:
52         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
53
54         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
55         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
56         min_action_window_duration}``); the model loads lazily on first use.
57         """
58
59         def __init__(self, config: Dict[str, Any]):
60             self.config = config
61             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
62             # `self.model([tensor])` inference call type-checks without a quarantine.
63             self.model: Any = None
64             self.device: Optional[str] = None
65
66         def lazy_load_model(self) -> None:
67             if self.model is not None:
68                 return
69
70             # Imported lazily so the module imports without torchvision present and so
71             # the test suite (which pre-sets self.model) never pulls in model weights.
72             from torchvision.models import detection as tv_detection
73
74             idx_cfg = self.config.get("indexing", {})
75             use_gpu = idx_cfg.get("use_gpu", True)
76             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
77             if use_gpu and not torch.cuda.is_available():
78                 logger.warning("use_gpu is True but CUDA is not available. Falling back to

```

```

CPU.")
78
79     model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80     if model_name not in _DETECTOR_FACTORIES:
81         logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82         model_name = _DEFAULT_DETECTOR
83
84     builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85     logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86     # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87     self.model = builder(weights="DEFAULT")
88     self.model.eval()
89     self.model.to(self.device)
90
91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)
99
100     def detect_and_track(self, frame: Any, tracker: Any,
101                          score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's
107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
114         with torch.no_grad():
115             outputs = self.model([tensor])
116
117         out = outputs[0]
118         boxes = out["boxes"].detach().cpu().numpy()
119         labels = out["labels"].detach().cpu().numpy()
120         scores = out["scores"].detach().cpu().numpy()
121
122         # Keep only confident person detections.
123         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
124         if not mask.any():
125             return []
126
127         # ByteTrack requires confidence to be populated (it filters on it internally).
128         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):
133             tid = tracked.tracker_id[i]
134             if tid is None:
135                 continue
136             x1, y1, x2, y2 = tracked.xyxy[i]
137             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))

```

```

138         return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                         velocity_thresh: float, merge_gap: float,
142                                         frame_skip: int = 3,
143                                         chunk_index: Optional[int] = None) ->
144     List[Dict[str, Any]]:
145         """
146         Runs person detection + tracking on a video chunk and extracts high-motion
147         action windows. Returns a list of dicts in the full video's timeframe, each
148         with:
149             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
150             track_id_count, chunk_index.
151
152         Args:
153             frame_skip: Process every Nth frame (1 = every frame). Higher values
154                         dramatically reduce inference cost with minimal quality loss.
155             chunk_index: Index of the source chunk (recorded on each window for
156             traceability).
157         """
158         cap = cv2.VideoCapture(chunk_path)
159         if not cap.isOpened():
160             logger.error(f"Could not open {chunk_path}")
161             return []
162
163         fps = cap.get(cv2.CAP_PROP_FPS)
164         if fps <= 0:
165             fps = 30.0
166
167         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
168         if frame_width <= 0:
169             frame_width = 1280.0 # Sensible fallback
170
171         # Effective sampling rate after frame-skipping
172         effective_fps = fps / max(frame_skip, 1)
173
174         # A fresh tracker per chunk ? track IDs only need to be consistent within a
175         # chunk (windows are computed per chunk, then merged across chunks by time).
176         tracker = self.make_tracker(effective_fps)
177         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
178         0.5)
179
180         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
181         # so windows can be enriched with motion stats for logging + fine-tuning.
182         active_frames = []
183         frame_idx = 0
184         track_history: Dict[int, Tuple[float, float, float, float]] = {}
185
186         while True:
187             ret, frame = cap.read()
188             if not ret:
189                 break
190
191             # Skip frames for performance ? at 30fps, every 3rd frame still
192             # gives ~10 samples/second, plenty for human-scale velocity.
193             if frame_idx % max(frame_skip, 1) != 0:
194                 frame_idx += 1
195                 continue
196
197             # Detect persons + assign persistent track IDs.
198             detections = self.detect_and_track(frame, tracker, score_thresh)
199
200             frame_peak_velocity = 0.0
201             frame_active_ids = set()
202             current_ids = set()

```

```

199
200         for track_id, bbox in detections:
201             current_ids.add(track_id)
202
203             if track_id in track_history:
204                 prev_bbox = track_history[track_id]
205                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207                 if velocity > velocity_thresh:
208                     frame_peak_velocity = max(frame_peak_velocity, velocity)
209                     frame_active_ids.add(track_id)
210
211                 track_history[track_id] = bbox
212
213             # Prune stale IDs that are no longer visible ? prevents false
214             # velocity spikes when the tracker reassigns an old ID.
215             track_history = {k: v for k, v in track_history.items() if k in current_ids}
216
217             if frame_active_ids:
218                 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
219
220             frame_idx += 1
221
222         cap.release()
223
224         # Group active frames into continuous windows
225         if not active_frames:
226             return []
227
228         max_gap_frames = int(merge_gap * fps)
229
230         def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
231             velocities = [v for _, v, _ in group]
232             track_ids = set().union(*[ids for _, _, ids in group])
233             return {
234                 "start": group[0][0] / fps + chunk_start,
235                 "end": group[-1][0] / fps + chunk_start,
236                 "active_frame_count": len(group),
237                 "peak_velocity": max(velocities),
238                 "mean_velocity": sum(velocities) / len(velocities),
239                 "track_id_count": len(track_ids),
240                 "chunk_index": chunk_index,
241             }
242
243         windows = []
244         group = [active_frames[0]]
245         for af in active_frames[1:]:
246             if af[0] - group[-1][0] <= max_gap_frames:
247                 group.append(af)
248             else:
249                 windows.append(_finalize(group))
250                 group = [af]
251         windows.append(_finalize(group))
252
253         # Filter out extremely short action windows
254         min_window_duration = self.config.get("indexing",
255 {}).get("min_action_window_duration", 2.0)
256         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
257
258         return windows
259
260     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
261                             frame_skip: int = 3) -> Dict[str, Any]:
262         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
263         video.

```

```

261
262         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
263         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
discards the
264         per-frame boxes), this returns the raw per-frame detections so the fusion
features can
265         align them with the shuttle trajectory. It reads the source by **absolute frame
index**
266         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
267         the WASB trajectory's ``.frame`` exactly ? that alignment is what
268         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
269
270         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...], "fps", "frame_width",
271         "frame_height"}}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
seek
272         accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
273         treat the indices as ±a few frames on real footage ? fine for window-level
features.
274         """
275         self.lazy_load_model()
276         cap = cv2.VideoCapture(video_path)
277         if not cap.isOpened():
278             logger.error(f"Could not open {video_path}")
279             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
280
281         fps = cap.get(cv2.CAP_PROP_FPS)
282         if fps <= 0:
283             fps = 30.0
284         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
285         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
286         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
287         step = max(frame_skip, 1)
288         tracker = self.make_tracker(fps / step)
289
290         start_frame = max(0, int(start_frame))
291         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
292         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
293         frame_idx = start_frame
294         while frame_idx <= end_frame:
295             ret, frame = cap.read()
296             if not ret:
297                 break
298             if (frame_idx - start_frame) % step == 0:
299                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
300                 frame_idx += 1
301         cap.release()
302         return {"frames": frames, "fps": fps,
303                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
304
305     def detections_over_windows(self, video_path: str, windows: List[Tuple[int, int]],
306                                frame_skip: int = 3,
307                                progress_cb: Optional[Callable[[int, int], None]] = None
308                                ) -> List[Dict[str, Any]]:
309         """Per-window person detections via ONE sequential pass over the video.
310
311         Opens the video ONCE and reads forward ? ``grab()`` to advance, ``retrieve()`` +
312         detect only on sampled frames inside a window ? instead of
``detections_per_frame``'s
313         per-window open + ``POS_FRAMES`` seek, which on a long video degrades to
314         decode-from-start per window (0(window_index) ? the fusion-P3 extraction
bottleneck;
315         172 windows took ~80 min). ``windows`` = ``[(start_frame, end_frame), ...]``;

```

```

returns a
316         list aligned to it, each ``{"frames": {idx: [(tid, bbox)]}, "fps", "frame_width",
317         "frame_height"}`` (same shape as ``detections_per_frame``). Each window gets its
own
318         window-local ByteTrack. ``progress_cb(frames_done, total_frames)`` (called every
~256
319         frames) drives progress logging.
320         """
321         self.lazy_load_model()
322         cap = cv2.VideoCapture(video_path)
323         n = len(windows)
324         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
325         if fps <= 0:
326             fps = 30.0
327         fw = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
328         fh = cap.get(cv2.CAP_PROP_FRAME_HEIGHT) if cap.isOpened() else 0.0
329         base = {"fps": fps, "frame_width": float(fw), "frame_height": float(fh)}
330         if not cap.isOpened() or n == 0:
331             cap.release()
332             return [dict(frames={}, **base) for _ in range(n)]
333
334         step = max(frame_skip, 1)
335         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
336         frames_per_win: List[Dict[int, Any]] = [{ } for _ in range(n)]
337         trackers = [self.make_tracker(fps / step) for _ in range(n)]
338         max_end = max(ef for _, ef in windows)
339
340         frame_idx = 0
341         while frame_idx <= max_end:
342             if not cap.grab():
343                 break
344             # Windows that want THIS frame (inside the range AND on that window's
sampling grid).
345             wis = [wi for wi, (sf, ef) in enumerate(windows)
346                   if sf <= frame_idx <= ef and (frame_idx - sf) % step == 0]
347             if wis:
348                 ok, frame = cap.retrieve()
349                 if ok:
350                     for wi in wis:
351                         frames_per_win[wi][frame_idx] = self.detect_and_track(
352                             frame, trackers[wi], score_thresh)
353             frame_idx += 1
354             if progress_cb is not None and frame_idx % 256 == 0:
355                 progress_cb(frame_idx, max_end)
356             cap.release()
357             return [dict(frames=frames_per_win[wi], **base) for wi in range(n)]
358

```

5. user (2026-06-14T06:32:20.802Z)

```

1         """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3         NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4         math is unit-testable in a GPU-free dev container. The detection (person boxes,
5         shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6         module only turns those signals into a handful of scale/translation-invariant
7         features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8         memorise this court/camera and fail to generalise off a small labelled set).
9
10        Conventions:
11        - A person box is ``(x1, y1, x2, y2)`` in pixels; ``(track_id, box)`` when motion
needs it.
12        - per_frame is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video

```

```

frame
13     index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14     - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15     - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;

```



```

70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
(0,0)."""
72         if court_polygon is None:
73             return True
74         if frame_width <= 0 or frame_height <= 0:
75             return False
76         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
77
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                        frame_height: float,
81                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
85             for _frame_idx, dets in sorted(per_frame.items())
86         ]
87
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
104         if not counts:
105             return 0.0
106         return sum(1 for c in counts if c >= min_players) / len(counts)
107
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                           frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,

```

```

131         frame_height: float, net: Optional[Net],
132         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137     0.0.
138     """
139     if net is None or not per_frame:
140         return 0.0
141     axis, pos = net
142     both = 0
143     for _frame_idx, dets in per_frame.items():
144         near, far = False, False
145         for _tid, box in dets:
146             if not person_in_court(box, frame_width, frame_height, court_polygon):
147                 continue
148             fx, fy = _foot_norm(box, frame_width, frame_height)
149             coord = fx if axis == "x" else fy
150             if coord < pos:
151                 near = True
152             else:
153                 far = True
154             if near and far:
155                 both += 1
156     return both / len(per_frame)
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                    pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                               frame_width: float, frame_height: float,
170                               pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
173     ~0**
174     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
175     """
176     if not per_frame or frame_width <= 0 or frame_height <= 0:
177         return 0.0
178     aligned = 0
179     colocated = 0
180     for p in shuttle_points or []:
181         if not getattr(p, "visible", False):
182             continue
183         dets = per_frame.get(int(p.frame))
184         if dets is None:
185             # shuttle frame wasn't person-sampled ? can't judge
186             continue
187         aligned += 1
188         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
189               for _tid, box in dets):
190             colocated += 1
191     return colocated / aligned if aligned else 0.0

```

```

192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                 frame_width: float, frame_height: float,
202                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                 net: Optional[Net] = None,
204                                 min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216

```

6. assistant (2026-06-14T06:32:25.416Z)

Now let me look at how the person cue is consumed in the segmenter/scorer path. Let me find the fusion segmenter, the default segmenter config, and `distill\_local.FEATURE\_NAMES`.

7. user (2026-06-14T06:32:26.489Z)

```

Found 20 files
docs\QUALITY_ITERATIONS.md
tests\test_fusion_compare.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\eval\fusion_golden.py
backend\pipeline\detectors\person_detector.py
tests\test_person_detector.py
docs\ANALYZERS_AND_RECONCILERS.md
tests\test_audio_features.py
backend\pipeline\detectors\audio_features.py
tests\test_fusion_hybrid.py
tests\test_fusion_features.py
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
backend\pipeline\detectors\fusion_features.py
backend\pipeline\segmenters\fusion_hybrid.py
mypy.ini
tests\test_yolo_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py

```

8. user (2026-06-14T06:32:27.831Z)

```

config.json:26:         "default_segmenter": "gemini",
README.md:74:This outputs a validation summary and indexes all rallies directly into the local
SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The

```

```

shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently
produces 0 segments and the observability report flags `silent_provider_noop`). Set it to
`yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
README.md:142:      "default_segmenter": "gemini",
README.md:180:*   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
README.md:242:   *Or set `default_segmenter`: "yolo_hybrid" (or `gemini`/`motion`) under
`indexing` in `config.json`.
README.md:246:[Omitted long matching line]
README.md:264:      "default_segmenter": "gemini",
run_process_and_stitch.py:74:      seg_name = cfg.indexing.default_segmenter
docs\CODE_MAP.md:31:  -> seg_name = req.segmenter or global_config.indexing.default_segmenter
(fallback 'motion')
docs\CODE_MAP.md:89:  - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
docs\CODE_MAP.md:99:[Omitted long matching line]
docs\CODE_MAP.md:105:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
docs\CODE_MAP.md:114:[Omitted long matching line]
docs\CODE_MAP.md:150:[Omitted long matching line]
docs\CODE_MAP.md:186:[Omitted long matching line]
docs\CODE_MAP.md:250:- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
docs\CODE_MAP.md:276:| Change a config default / add a config field |
`@backend/config/models.py` (single source of truth) -> regenerate via
`setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
docs\EXPERIMENT_HARNESS.md:106:- `backend/config/models.py::IndexingConfig` ?
`default_segmenter` (:65, default `yolo_hybrid`), `detector_model` (:72),
`detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75),
`min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). **No
`min_crossings` knob yet** (see fusion P2).
docs\EXPERIMENT_HARNESS.md:123:JSON-serializable; selected via the existing `segmenter_registry`
+ `IndexingConfig.default_segmenter`.
docs\EXPERIMENT_HARNESS.md:155:pinned ? **rollback = flip `default_segmenter`/variant config,
never a code revert.**
docs\EXPERIMENT_HARNESS.md:185:3. **Per-cue enable flags** in `IndexingConfig` (default
`False`); `default_segmenter` already exists.
docs\MULTI_SIGNAL_FUSION_PLAN.md:28:| `yolo_hybrid` (the current `default_segmenter`) | person
velocity (torchvision + ByteTrack) | `backend/pipeline/segmenters/yolo_hybrid.py` |
docs\MULTI_SIGNAL_FUSION_PLAN.md:271:  **default-OFF** ? `default_segmenter` unchanged),
subclassing the trajectory engine via two identity
docs\MULTI_SIGNAL_FUSION_PLAN.md:274:  and ? when `indexing.fusion.cue_flags.person` is on ?
runs the person cue over each window's frames
backend\config\models.py:82:  shuttle net-crossing count (Gate A signal) and ? only when
`cue_flags['person']` is
backend\config\models.py:94:  # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF
unless explicitly
backend\config\models.py:96:  cue_flags: dict[str, bool] = Field(default_factory=dict)
backend\config\models.py:112:  default_segmenter: str = "yolo_hybrid"
backend\config\models.py:144:  @field_validator("default_segmenter")
docs\QUALITY_ITERATIONS.md:139:**Shipping shape.** A NEW `fusion` segmenter, **default OFF**
(`default_segmenter` unchanged); Gate A
tests\test_config.py:23:  assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:48:  assert f.cue_flags == {} # person cue OFF
tests\test_config.py:61:  "cue_flags": {"person": True}, "court_polygon": [[0.1,
0.1], [0.9, 0.1], [0.5, 0.9]],
tests\test_config.py:67:  assert f.cue_flags == {"person": True}
tests\test_config.py:78:  "default_segmenter": "gemini",
tests\test_config.py:89:  assert cfg.indexing.default_segmenter == "gemini"
tests\test_config.py:102:  "indexing": {"default_segmenter": "yolo_hybrid"},
tests\test_config.py:110:  assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:138:  "indexing": {"default_segmenter": "gemini", "wasb":

```

```

{"velocity_threshold": 0.02}},
tests\test_config.py:154:         assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:198:         assert cfg.get("indexing", {}).get("default_segmenter") ==
"yolo_hybrid"
docs\TRACKNET_WSL_SETUP.md:71:Then select it as the segmenter (e.g. set
`indexing.default_segmenter` to
backend\eval\experiment.py:86:         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ?
informational provenance
backend\eval\experiment.py:110:         cue_flags: Dict[str, bool] = field(default_factory=dict)
backend\eval\experiment.py:132:         "cue_flags": self.cue_flags,
backend\main.py:148:         default_seg = getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
backend\main.py:712:         seg_name = args.segmenter or getattr(getattr(global_config,
"indexing", None), "default_segmenter", "motion")
tests\test_experiment.py:58:         v = Variant(name="gateA", min_crossings=2, cue_flags={"person":
False})
tests\test_experiment.py:81:         disabled = Variant(name="fusion_off", cue_flags={"person":
False, "audio": False},
backend\pipeline\segmenters\fusion_hybrid.py:10: is explicitly enabled
(``indexing.fusion.cue_flags.person``), the person-cue features
backend\pipeline\segmenters\fusion_hybrid.py:88:         if fcfg.get("cue_flags",
{}).get("person", False):
backend\pipeline\segmenters\fusion_hybrid.py:142:         person_on = bool(fcfg.get("cue_flags",
{}).get("person", False))
backend\reporting\spec.yaml:133:         The segmenter that ran differs from config.json's
default_segmenter, and no
tests\test_fusion_hybrid.py:6:persistence keeps the superset; and the person cue, when enabled,
adds the person features.
tests\test_fusion_hybrid.py:98:def test_person_cue_adds_features_when_enabled():
tests\test_fusion_hybrid.py:111:         db, _provider, _res = _run(indexing={"fusion":
{"cue_flags": {"person": True}}})
tests\test_fusion_hybrid.py:133:         db, provider, _res = _run(indexing={"fusion":
{"cue_flags": {"person": True}, "min_players": 2}})
docs\ANALYZERS_AND_RECONCILERS.md:204:Flag-gated, **default OFF** via the existing
`indexing.fusion.cue_flags` (a sibling `analyzer_flags`
docs\ANALYZERS_AND_RECONCILERS.md:507: (none enabled ? byte-identical). Retrofit
`net_crossings` + person features as the first two analyzers.
backend\reporting\harvest.py:216:         config_default = (config.get("indexing") or
{}).get("default_segmenter")
tests\test_reporting_harvest.py:46:         config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
tests\test_reporting_harvest.py:122:         config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "gemini"}},
tests\test_reporting_harvest.py:137:         config={"indexing": {"default_segmenter":
"motion"}})
tests\test_reporting_harvest.py:146:         cfg = {"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}}
tests\test_reporting_harvest.py:189:         config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}},

```

9. assistant (2026-06-14T06:32:32.600Z)

The default segmenter is `yolo\_hybrid` (model default) / `gemini` (config.json) ? NOT `fusion`. Let me read the fusion segmenter and the scorer (distill_local / fusion_compare) to verify how the person features enter the decision.

10. user (2026-06-14T06:32:33.069Z)

```

1         """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3         Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4         TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5         do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6         shuttle-seeded window via two overridable seams the engine exposes:
7

```

```

8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9       GPU-free, computed from the trajectory we already have); and, only when the person cue
10      is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11      (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12      fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13      torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14      model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15      via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17      (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18      dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19      **Persisted candidates remain the full superset** regardless, so the offline
experiment
20      harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property

```

```

59     def description(self) -> str:
60         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                 "AI provider sets semantic boundaries.")
63
64     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66         if substrate == "tracknet":
67             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68             return TrackNetRunner(cfg), {
69                 "weights_path": cfg.weights_path,
70                 "queue_length": cfg.queue_length,
71                 "fusion_substrate": "tracknet",
72             }
73         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                frame_width: float, video_path: str,
78                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                               min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                          frame_width: float, video_path: str,
105                          fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""

```

```

109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150             keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

11. user (2026-06-14T06:32:33.903Z)

```

70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0

```



```

76
77
78 class FusionConfig(BaseModel):
79     """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
80     P2).
81
82     Every default reproduces control behaviour: the fusion segmenter persists the
83     shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
84     is
85     true ? the person-cue features, but it does NOT gate the served handoff unless a
86     threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
87     mask
88     is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
89     only);
90     supplied ? off-court persons (spectators / adjacent courts) are excluded.
91     """
92     # Which trajectory substrate seeds the windows (shuttle stays primary).
93     substrate: Literal["wasb", "tracknet"] = "wasb"
94     # Windowing velocity threshold for the fusion family (the engine reads
95     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
96     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
97     velocity_threshold: float = 0.02
98     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
99     # enabled ? so the default path never imports torch / touches the GPU.
100    cue_flags: dict[str, bool] = Field(default_factory=dict)
101    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
102    handoff.
103    # 0 = OFF (no gating; persisted candidates are always the full superset for offline
104    re-scoring).
105    min_crossings: int = Field(default=0, ge=0)
106    # Served Gate B: when the person cue is on, drop windows with median in-court
107    players
108    # < this. 0 = OFF.
109    min_players: int = Field(default=0, ge=0)
110    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
111    court_polygon: Optional[list[list[float]]] = None
112    # Net line for the person two-sided-motion split (person features only ? the shuttle
113    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
114    net_axis: Literal["x", "y"] = "y"
115    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
116
117 class IndexingConfig(BaseModel):
118     default_segmenter: str = "yolo_hybrid"
119     use_gpu: bool = True
120     motion_threshold: float = 0.08
121     min_segment_duration: float = 3.0
122     dead_time_merge_gap: float = 2.0
123     chunk_duration: float = 90.0
124     chunk_overlap: float = 10.0
125     detector_model: str = "fasterrcnn_resnet50_fpn"
126     detector_score_threshold: float = 0.5
127     yolo_velocity_threshold: float = 0.15
128     yolo_frame_skip: int = 3
129     yolo_tracker: str = "bytetrack.yaml"
130     yolo_prefetch_workers: int = 2
131     min_action_window_duration: float = 2.0
132     log_yolo_candidates: bool = True
133     store_yolo_candidates: bool = True
134     ai_handoff_padding: float = 2.0
135     ai_max_workers: int = 5
136     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
137     # chunks of a high-bitrate (4K) source blow past the provider upload cap
138     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
139     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate

```

```

134     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
135     # Matches gemini_refine's --clip-scale-width default.
136     ai_chunk_scale_width: int = 1280
137     # Optional debug knob: trim every video to the first N seconds before processing.
138     debug_duration: Optional[float] = None
139
140     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
141     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
142     fusion: FusionConfig = Field(default_factory=FusionConfig)
143
144     @field_validator("default_segmenter")
145     @classmethod
146     def segmenter_must_be_registered(cls, v: str) -> str:
147         # Registry check happens at runtime in main/segmenter selection.
148         # Here we just allow any string for forward compatibility.
149         return v
150
151     @model_validator(mode="after")
152     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
153         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
154         # step makes `while t < duration: t += step` loop forever, growing memory before
any
155         # GPU work. Reject the bad config at load time instead.
156         if self.chunk_overlap >= self.chunk_duration:
157             raise ValueError(
158                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
159                 f"({self.chunk_duration}); otherwise chunking never advances."

```

12. assistant (2026-06-14T06:32:37.240Z)

Now let me look at the learned scorer to verify the "no special-casing" and "retained" criteria. Let me find `distill\_local.FEATURE\_NAMES` and how `fusion\_compare`/`fusion\_golden` score the features.

13. user (2026-06-14T06:32:39.233Z)

```

6:designed seam): the person features are extra columns the learned LogisticRegression weights,
22:from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
27:    (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
45:        for fn in SHUTTLE_FEATURE_NAMES:
47:        for fn in PERSON_FEATURE_NAMES:
57:    shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
60:        feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),

```

14. user (2026-06-14T06:32:39.355Z)

backend\eval\distill_local.py

15. user (2026-06-14T06:32:42.010Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's

```

```

6      designed seam): the person features are extra columns the learned LogisticRegression
weights,
7      so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9      Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58                threshold=threshold)
59         fusion = experiment.Variant(name="shuttle_person_fusion",
60                feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61                threshold=threshold)
62         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64

```

```

65     def format_result(res: Dict[str, Any], iou: float) -> str:
66         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
67         ds = delta_stats(res)
68         st = ds["sign_test"]
69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
74             f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
75             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
76             f"| CI clears 0: {ds['ci_excludes_zero']}",
77             "  per-video ?F1:",
78         ]
79         for pv in res["per_video_delta"]:
80             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
81         return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

16. user (2026-06-14T06:32:42.532Z)

```

1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).

```

```

10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31
32     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
33     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
34     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
35     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
36
37
38     def _derive_video_path(spec: Dict[str, Any]) -> str:
39         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
40         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
41         if spec.get("video_path"):
42             return str(spec["video_path"])
43         labels = str(spec["labels"])
44         if labels.endswith(".rallies.csv"):
45             return labels[: -len(".rallies.csv")] + ".MP4"
46         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
47
48
49     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
50                     frame_skip: int = 3, iou: float = 0.5,
51                     log_every_sec: float = 15.0) -> Dict[str, Any]:
52         """One golden video -> a feature record (shuttle + person features per candidate
window).
53
54         Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle{}, person{}, label}], gts}``.
55         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
56         when None a real `PersonDetector` is built (loads torchvision on first use).
57         """
58         fps, fw = float(spec["fps"]), float(spec["frame_width"])
59         traj = str(spec["trajectory"])
60         points = TrackNetRunner.parse_trajectory_csv(traj)
61         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
62         gts = load_golden_gts(str(spec["labels"]))
63         labels = label_candidates(intervals, gts, iou)
64         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
65         # (parity with the shuttle net-crossing gate) rather than hard-coding an

```

```

orientation.
66     axis, net_px, _span = rally_gate.estimate_play_axis(points)
67     video_path = _derive_video_path(spec)
68
69     if detector is None:
70         from backend.pipeline.detectors.person_detector import PersonDetector
71         detector = PersonDetector({"indexing": {"use_gpu": True}})
72
73     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
74     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
75     n = len(intervals)
76     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
77     print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
78     t0 = time.monotonic()
79     state = {"last": t0}
80
81     def _progress(done: int, total: int) -> None:
82         now = time.monotonic()
83         if now - state["last"] >= log_every_sec or done >= total:
84             el = now - t0
85             rate = done / el if el > 0 else 0.0
86             eta = (total - done) / rate if rate > 0 else 0.0
87             pct = 100 * done // total if total else 0
88             print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
89                 f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
90             state["last"] = now
91
92     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
93                                                  progress_cb=_progress)
94
95     candidates: List[Dict[str, Any]] = []
96     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
97         det = per_window[i]
98         det_fw = det.get("frame_width") or fw
99         det_fh = det.get("frame_height") or 0.0
100         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
101         # normalisation fusion_features uses (x/width, y/height).
102         if axis == "x":
103             net_norm = net_px / det_fw if det_fw > 0 else 0.5
104         else:
105             net_norm = net_px / det_fh if det_fh > 0 else 0.5
106         sf, ef = win_frames[i]
107         shuttle_in = [p for p in points if sf <= p.frame <= ef]
108         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
109                                           court_polygon=None, net=(axis, net_norm))
110         candidates.append({
111             "start": float(s), "end": float(e),
112             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
113             "person": {k: float(v) for k, v in person.items()},
114             "label": int(labels[i]),
115         })
116     return {
117         "name": spec["name"], "fps": fps, "frame_width": fw,
118         "candidates": candidates,
119         "gts": [[float(a), float(b)] for a, b in gts],
120     }
121
122
123 def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
124        fresh: bool = False, smallest_first: bool = False) -> List[str]:
125     with open(manifest_path, encoding="utf-8") as f:
126         specs = json.load(f)

```

```

127         if smallest_first:
128             # Process small videos first for quick feedback (the slow full match lands
last);
129             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
130             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
131             if os.path.isfile(s.get("trajectory", ""))
else 0))
132         os.makedirs(out_dir, exist_ok=True)
133         written: List[str] = []
134         for vi, spec in enumerate(specs, 1):
135             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
136             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
137             # run left off instead of re-detecting). --fresh forces a redo.
138             if not fresh and os.path.isfile(out):
139                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
140                       f"-> {out} (use --fresh to redo)", flush=True)
141                 written.append(out)
142                 continue
143             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
144             t0 = time.monotonic()
145             data = extract_video(spec, frame_skip=frame_skip, iou=iou)
146             n_pos = sum(c["label"] for c in data["candidates"])
147             with open(out, "w", encoding="utf-8") as f:
148                 json.dump(data, f, indent=2)
149             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
150                   f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
151                   f"-> {out}", flush=True)
152             written.append(out)
153         return written
154
155
156     def main() -> None:
157         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
158         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
159         ap.add_argument("--out-dir", default="output")
160         ap.add_argument("--frame-skip", type=int, default=3)
161         ap.add_argument("--iou", type=float, default=0.5)
162         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
163         ap.add_argument("--smallest-first", action="store_true",
164                           help="process smaller videos first for quick feedback (LOVO is
order-independent)")
165         args = ap.parse_args()
166         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
167             smallest_first=args.smallest_first)
168
169
170     if __name__ == "__main__":
171         main()
172

```

17. assistant (2026-06-14T06:32:46.421Z)

Now let me verify the experiment harness's `Variant` and `compare` to confirm the features enter as learned columns (LogisticRegression), not a hard if/else. And confirm `distill\_local.FEATURE\_NAMES` exists.

18. user (2026-06-14T06:32:47.419Z)

```

1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`    ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
56     ratio
57     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
58     scores
59
60     logger = logging.getLogger(__name__)
61
62     # Where a comparison run appends one reproducible record. Tracked location (NOT under
63     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64     # regression.DEFAULT_BASELINE_PATH.
65     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")

```



```

64     _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67     class ExperimentError(ValueError):
68         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69         score an unlabeled video with no pinned model, or a gate referencing an absent
70         feature)."""
71
72
73     # ----- Variant
74
75
76     @dataclass
77     class Variant:
78         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80         A variant turns one video's persisted candidate windows + features into rally
81         intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82         filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83         to control ? the core no-regression guarantee.
84
85         Fields:
86         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87           for the leaderboard and for selecting the served path (the existing
88           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89           New cues are recorded here default-OFF.
90         - ``min_crossings`` ? decision-gate **Gate A** : keep only windows whose persisted
91           ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94         - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95           whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96           cue persists that feature).
97         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99           required for ``compare_unlabeled`` on a learned variant.
100        - ``feature_names`` ? the persisted features the learned filter consumes. When set
101          WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102          (honest) ? the recipe, not a pinned artifact.
103        - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104          overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105        """
106
107        name: str
108        segmenter: str = "wasb_hybrid"
109        config_overrides: Dict[str, Any] = field(default_factory=dict)
110        cue_flags: Dict[str, bool] = field(default_factory=dict)
111        min_crossings: int = 0
112        min_players: int = 0
113        classifier_model: Optional[str] = None
114        feature_names: Optional[List[str]] = None
115        threshold: float = 0.5
116        merge_gap: float = 1.0
117        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119        # failure where a fixed 0.5 zeroed out a small-candidate video.
120        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121        calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123        def is_learned(self) -> bool:
124            """True if this variant applies a learned classifier (pinned model or

```

```

recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
151         the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154         return sha1(blob.encode()).hexdigest()[:12]
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)

```

```

188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196         cols = [_feature_column(vc, name) for name in feature_names]
197         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
202         n = len(vc.intervals)
203         mask = [True] * n
204         if variant.min_crossings > 0:
205             col = _feature_column(vc, "net_crossings")
206             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
207         if variant.min_players > 0:
208             col = _feature_column(vc, "players_in_court")
209             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
210         return mask
211
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
225
226         """
227         mask = _gate_mask(variant, vc)
228         if variant.feature_names:
229             if model is None:
230                 if variant.classifier_model is None:
231                     raise ExperimentError(
232                         f"variant {variant.name!r} is learned but has no model to predict "
233                         f"with (pass a fitted model or set classifier_model)")
234                 model = LogisticRegression.load(variant.classifier_model)
235                 proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
236             if calibrator is not None:
237                 proba = [calibrator(p) for p in proba]
238             thr = variant.threshold if threshold is None else threshold
239             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
240             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
241             return merge_overlaps(kept, gap_tol=variant.merge_gap)
242
243     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                             train_videos: Sequence[VideoCandidates], iou: float
245                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None

```

```

248         (use the variant default). Honest: never looks at the held-out video.
249         """
250     calibrator: Optional[Callable[[float], float]] = None
251     if variant.calibrate:
252         raw: List[float] = []
253         y: List[int] = []
254         for vc in train_videos:
255             raw.extend(float(p) for p in
256                         model.predict_proba(_feature_matrix(vc, variant.feature_names or
257                                                         [])))
258             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
259         if variant.calibrate == "platt":
260             calibrator = fit_platt(raw, y)
261         elif variant.calibrate == "isotonic":
262             calibrator = fit_isotonic(raw, y)
263         else:
264             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
265 'platt'/'isotonic')")
266     threshold: Optional[float] = None
267     if variant.tune_threshold:
268         best_t, best_f1 = variant.threshold, -1.0
269         for k in range(1, 20):
270             # grid 0.05..0.95 on the train
271             t = round(0.05 * k, 2)
272             f1s = [score(predict(variant, vc, model=model, threshold=t,
273                                 calibrator=calibrator),
274                       vc.gts or [], iou).f1 for vc in train_videos]
275             mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
276             if mean_f1 > best_f1:
277                 best_f1, best_t = mean_f1, t
278             threshold = best_t
279     return calibrator, threshold
280
281 # ----- variant scoring
282
283 def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
284     """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
285 the
286 evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
287     X: List[List[float]] = []
288     y: List[int] = []
289     for vc in videos:
290         if vc.gts is None:
291             raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
292         X.extend(_feature_matrix(vc, variant.feature_names or []))
293         y.extend(label_candidates(vc.intervals, vc.gts, iou))
294     if not X:
295         raise ExperimentError("cannot train a learned variant on zero candidates")
296     return LogisticRegression().fit(X, y, variant.feature_names)
297
298 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
299                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
300     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
301     mean aggregate.
302
303     Prediction policy:
304     - **static variant** (no learned filter): predict each video directly ? no
305     training,
306     so no leakage; per-video scoring is already honest.
307     - **learned, pinned model** (`classifier_model` set): score every video with the
308     SHIPPED model. ? optimistic if those videos were in its training set ? use this

```

```

to
306         report "how the shipped artifact does here", not for the promotion decision.
307     - **learned recipe** (`feature_names`, no pinned model) + ``honest=True``: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309     - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310     """
311     for vc in videos:
312         if vc.gts is None:
313             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315     per_video: List[Dict[str, Any]] = []
316     predictions: Dict[str, List[Interval]] = {}
317
318     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319     pinned_model = (LogisticRegression.load(variant.classifier_model)
320                     if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                  for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359 def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360     """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare

```

```

mean)."""
361     return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS $6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404               iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (``EXPERIMENT_HARNESS.md`` $5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing ``metrics.score`` (per video) +
``calibration.pct_improvement``;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         ``record_experiment`` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap

```

```

CIs, a
413     paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414     existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415     shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416     bare LOVO mean at n?6 is not trustworthy on its own.
417     """
418     if len(videos) < 1:
419         raise ExperimentError("compare needs at least one labeled video")
420     eval_a = evaluate_variant(videos, variant_a, iou, honest)
421     eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427     per_video_delta = [
428         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429         for p in eval_b["per_video"]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455         agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465             the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a
468         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469         ``metrics.match_intervals`` ? no new matching logic.

```

```

470     """
471     preds_a = predict(variant_a, video)
472     preds_b = predict(variant_b, video)
473     mr = match_intervals(preds_a, preds_b, agree_iou)
474
475     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476               for m in mr.matches]
477     agree_ious = [m.iou for m in mr.matches]
478     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
480
481     def _dur(ivs: List[Interval]) -> float:
482         return sum(e - s for s, e in ivs)
483
484     return {
485         "video": video.name,
486         "agree_iou": agree_iou,
487         "variant_a": variant_a.name,
488         "variant_b": variant_b.name,
489         "num_a": len(preds_a),
490         "num_b": len(preds_b),
491         "num_agreed": len(agreed),
492         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493         "duration_a": _dur(preds_a),
494         "duration_b": _dur(preds_b),
495         "agreed": agreed,
496         "a_only": a_only,
497         "b_only": b_only,
498     }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)

```



```

534         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535         with open(path, "w", encoding="utf-8") as f:
536             json.dump(entries, f, indent=2)
537         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                     row["variant_a"]["name"], row["variant_b"]["name"],
539                     (result.get("delta") or {}).get("f1", 0.0), path)
540         return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551         touching
552         function; the comparison itself is pure and offline. Exercised owner-side on a real
553         DB,
554         but works against any `Database` (including an in-memory one in tests).
555         """
556         rows = db.query_candidate_segments(video_id, detector=detector)
557         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558                                     rows]
559         features: Dict[str, List[float]] = {}
560         for name in feature_names:
561             if name == "duration":
562                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
563                                 rows]
564             else:
565                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
566                                 rows]
567         return VideoCandidates(name=video_id, intervals=intervals, features=features,
568                                gts=gts)
569

```

19. assistant (2026-06-14T06:32:51.919Z)

This confirms the core mechanism. Now let me verify the two remaining "retained" / non-significant claims: check the leaderboard for the recorded n=6 result, and confirm `distill\_local.FEATURE\_NAMES` exists. Let me also confirm `min\_players` (the only person-feature that appears in a hard gate) is default-OFF and is a separate served-gate knob, distinct from the learned-scorer column.

20. user (2026-06-14T06:32:53.936Z)

```

40: FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
41:                   "net_crossings"]
116:     model = LogisticRegression().fit(X, y, FEATURE_NAMES)
135:     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
137:     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))

```

21. user (2026-06-14T06:32:55.068Z)

```

d7b1f51 feat(eval): honest ?F1 ? bootstrap CI + paired sign test on the fusion lift (#127)
813d1f8 feat(eval): fusion_golden --smallest-first (quick-feedback extraction order) (#126)
351a27e perf(eval): single-pass person detection for fusion_golden (~10x faster extraction)
(#125)
f4d7008 feat(eval): fusion_golden progress logging + resumability (observability) (#123)
39e961b feat(eval): fusion P3 measurement ? golden person-feature extractor + LOVO compare
(#121)
cbf2236 feat(segmenters): FusionSegmenter ? shuttle + net-crossing + person-cue fusion (P2,

```

default-OFF) (#117)

df7d5df refactor(detectors): extract the person cue into PersonDetector (fusion P1) (#115)

22. user (2026-06-14T06:32:55.996Z)

```
[
  {
    "timestamp": "2026-06-14T06:23:51.507393+00:00",
    "iou": 0.5,
    "label_set_hash": "50d98ffbc370",
    "num_videos": 6,
    "variant_a": {
      "name": "shuttle_only",
      "spec_hash": "47c867c36f2f",
      "aggregate": {
        "precision": 0.31807603780927995,
        "recall": 0.3101947096042718,
        "f1": 0.3068611057861412,
        "mean_iou": 0.5791831889148886
      }
    },
    "variant_b": {
      "name": "shuttle_person_fusion",
      "spec_hash": "253f4709b4c4",
      "aggregate": {
        "precision": 0.3617228001510626,
        "recall": 0.2708939086387243,
        "f1": 0.2863398846047612,
        "mean_iou": 0.5690111885714347
      }
    },
    "delta": {
      "precision": 0.04364676234178266,
      "recall": -0.0393008009655475,
      "f1": -0.02052122118138,
      "mean_iou": -0.01017200034345389
    },
    "honest_delta_f1": {
      "mean_delta": -0.02052122118137999,
      "ci_low": -0.16462806145542527,
      "ci_high": 0.16224446866060574,
      "n": 6.0,
      "ci_excludes_zero": false,
      "sign_test": {
        "wins": 2.0,
        "losses": 4.0,
        "ties": 0.0,
        "n_effective": 6.0,
        "p_value": 0.6875
      }
    }
  },
  {
    "timestamp": "2026-06-14T06:24:56.487711+00:00",
    "iou": 0.5,
    "label_set_hash": "50d98ffbc370",
    "num_videos": 6,
    "variant_a": {
      "name": "shuttle_person",
      "spec_hash": "ee89f1e41604",
      "aggregate": {
        "precision": 0.3617228001510626,
        "recall": 0.2708939086387243,
        "f1": 0.2863398846047612,
        "mean_iou": 0.5690111885714347
      }
    }
  }
]
```

```

    }
  },
  "variant_b": {
    "name": "shuttle_person_audio",
    "spec_hash": "d68c39eba87c",
    "aggregate": {
      "precision": 0.4530798177857001,
      "recall": 0.3408138120839734,
      "f1": 0.3655826552797436,
      "mean_iou": 0.7074802852140344
    }
  },
  "delta": {
    "precision": 0.0913570176346375,
    "recall": 0.06991990344524907,
    "f1": 0.07924277067498242,
    "mean_iou": 0.13846909664259965
  },
  "honest_delta_f1": {
    "mean_delta": 0.07924277067498244,
    "ci_low": -0.053508315003560535,
    "ci_high": 0.2029045239922802,
    "n": 6.0,
    "ci_excludes_zero": false,
    "sign_test": {
      "wins": 4.0,
      "losses": 2.0,
      "ties": 0.0,
      "n_effective": 6.0,
      "p_value": 0.6875
    }
  }
}
]

```

23. assistant (2026-06-14T06:33:11.958Z)

This confirms the full picture. The $n=6$ measured ?F1 for the person cue is ~ -0.021 (CI does not exclude zero, sign test 2W/4L $p=0.6875$) ? non-significant. Yet the code is fully retained. Let me do two final checks: confirm nothing in git history deleted/disabled the person producer after the measurement, and confirm the served default path (``yolo_hybrid``) doesn't accidentally weight person features via a scorer.

24. user (2026-06-14T06:33:14.055Z)

```

8836dfe chore: scripts/ for run-drivers + CODE_MAP $0 file-placement guidelines (#138)
60074ae chore: scripts/ for run-drivers + CODE_MAP $0 file-placement guidelines
9eb06f4 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus
fa12829 chore: gitignore scratch/ to keep git status clean (#136)
d4e66c7 chore: gitignore scratch/ to keep git status clean
55df82f feat(obs): run-telemetry black box ? fps + GPU temp/clocks/power + CPU/RAM (rotating
~10KB JSONL) (#134)
a9432a9 feat(db): self-scheduling job substrate ? migration v3 + claim_due_job (P2) (#135)
5c7af00 feat(db): self-scheduling job substrate ? migration v3 + claim_due_job (P2)
bc40b83 feat(obs): run-telemetry black box ? fps + GPU temp/clocks/power + CPU/RAM (rotating
~10KB JSONL)
ec2002d feat(infra): ExecutionPolicy P1 ? bound the Gemini hang (op_timeout + bounded poll)
(#133)
---HEAD/branch---
docs/i3-fusion-measured
9eb06f4 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus

```

25. user (2026-06-14T06:33:15.652Z)

No matches found

26. assistant (2026-06-14T06:33:19.910Z)

The current branch is `docs/i3-fusion-measured` with a `docs(quality)` commit logging the measured ?F1. The `yolo\_hybrid` (default served segmenter) has zero references to person-cue features ? confirming no scorer in the served path weights them. Let me confirm the docs change only LOGGED the result (didn't disable code), and verify the person producer/feature math is intact on this branch.

27. user (2026-06-14T06:33:21.025Z)

```
commit 9eb06f4efbd10bbd57c82f998936506bd9ff177f
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Sun Jun 14 11:58:17 2026 +0530
```

docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus

Records the honest LOVO result now that the GPU person-feature extraction finished all 6 golden videos. +person (P3) ?F1=-0.021 (CI [-0.165,+0.162], 2W/4L) and +audio on top (P4 incremental) ?F1=+0.079 / +27.7% (CI [-0.054,+0.203], 4W/2L) -- BOTH CIs span 0 and both sign tests are non-significant at n=6, so neither cue is promotable; both stay default-OFF (promote-only-on-lift). Person adds noise (no court polygons -> players_in_court counts spectators); audio is the more promising lead (wins 4/6, endpoint 0.366 > shuttle-only 0.307) but n=6 can't confirm. Negative/uncertain results kept on purpose -- the honest ablation backbone. Recorded in eval_baselines/experiment_leaderboard.json.

Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>

```
docs/QUALITY_ITERATIONS.md | 19 +++++
1 file changed, 19 insertions(+)
```

28. user (2026-06-14T06:33:21.942Z)

```
docs/QUALITY_ITERATIONS.md | 19 +++++
1 file changed, 19 insertions(+)
```

29. assistant (2026-06-14T06:33:25.726Z)

Confirmed. The only change on this branch vs master is a docs-only addition to `QUALITY\_ITERATIONS.md` (19 lines added, +0 to code). The non-significant n=6 result was LOGGED, not acted on by deleting/disabling code. The person producer + feature math are fully intact.

I have all the evidence needed. Let me compile the audit findings.

30. user (2026-06-14T06:33:59.130Z)

Structured output provided successfully

31. assistant (2026-06-14T06:34:08.757Z)

Audit complete. Summary of findings for the PERSON cue:

- **present: TRUE** ? Producer (`PersonDetector`, `person\_detector.py:50`) and feature math (all 5 features + `compute\_person\_features`/`PERSON\_FEATURE\_NAMES` in `fusion\_features.py`) both exist and are functional.
- **default_off: TRUE** ? `cue\_flags` defaults to `{}` (person OFF, `models.py:96`); `default\_segmenter` is `yolo\_hybrid`/`gemini`, never `fusion`; enrichment is flag-gated (`fusion\_hybrid.py:88`) so the default path never loads the detector; the served Gate B `min\_players` defaults to 0.
- **no_special_casing: TRUE** ? The features enter the decision as extra **feature columns** a `LogisticRegression` weights (`experiment.py:234`, `fusion\_compare.py:57-61` differ only by `feature\_names`). No hard if/else on a person feature in the learned decision path.
- **retained: TRUE** ? n=6 ?F1 = ?0.021 (CI [?0.165, +0.162], 2W/4L, p=0.6875) is non-

significant, yet the only change on the current branch is a ****docs-only**** log (``9eb06f4``, +19 lines to ``QUALITY_ITERATIONS.md``, zero code). Producer + feature math intact; the persisted ``*_golden_features.json`` substrate makes it re-measurable as the corpus grows.

Key concerns flagged: the n=6 extraction used ``court_polygon=None`` so ``players_in_court`/`two_sided_motion`` counted spectators (the negative result is partly un-tuned features, not a dead cue); ``min_players`` is a separate hard-gate consumer of one person feature (default-OFF, distinct from the learned path); and the served person wiring (``fusion_hybrid._person_features``) is only mock-tested ? real-video verification is owner-side.